

Engineering is a structured approach. It helps to build something in a proper way. The main difference between scientists and engineers is that scientists research something without necessarily creating something from it, but engineers create something tangible.

65% of softwares start up and fail each year due to a lack of research.

Two types of software:

- **BTB** - Business to Business (for enterprises)
- **BTC** - Business to Consumer (for the public) eg. Coursera, Netflix

Software is built for different audience types. They also come in different models, like pay-once or SaaS. SaaS model is popular because you pay for what you use on a monthly basis and can cancel if you don't want/need to use it anymore. Cloud is SaaS too and is more convenient and way cheaper than a proper server room.

SOFTWARE DEVELOPMENT LIFE CYCLE

Development life cycle helps with every step of the process. **(SDLC)**

- **First stage: Requirements.** Requirement analysis/gathering. If the requirements aren't clear, it'll take more time due to reworks. Spend more time on requirements to avoid this. Assuming the client agrees, there will be a sign-off.
- **Second stage: Design.** Software needs an architecture, developed with high-level and low-level design. Specific, individual modules come under low-level designs. High-level is for the overall project.
- **Third stage: Development.** Developers write the code here. Test cases are written parallel to coding so that testers aren't just sitting idle until their stage.
- **Fourth stage: Testing.** Execution of testing can only start after coding has been completed. Testers find bugs for coders to fix. Testers think like an end-user and test the application like so.
- **Fifth stage: Implementation.** The code was in a dummy environment until now, but you have to make it properly usable now. They contain multiple environments that users cannot access, and each company has at least three of the following environments: Development, QA, Beta (or Pre-Production), Production. Need to ensure the product works for each specification of device, browser, etc.
- **Sixth stage: Maintenance.** Now you can add new features, fix bugs that have been reported after the product has gone live, etc. Three months warranty is given, then if something breaks, it can be fixed for free. Beyond that, they will not fix it. Depends on the budget. Some companies have dedicated support teams to ensure customer bugs are addressed, but any additional features that have been suggested should go through all the development stages once more.

There needs to be a **deliverable** for each stage:

1) Business analysts will discuss to see what the needs and budget are to determine if a project is possible. Otherwise, there could be a low budget and time is wasted, etc. They take the requirements from the customer and have internal discussions with the project/product manager. They then create a document that is called any of the following:

- SRS - Software requirement specification / SRD - Software requirement document
- BRS - Business (...)
- CRS - Customer (...)

What the document includes: What programming language needs to be used, how many software engineers/developers are required (designers, testers, etc.), what the project duration is, what the infrastructures needed are (such as cloud services).

- 2) Design document
- 3) Code
- 4) Test case execution results
- 5) Go live
- 6) Constant support

Cost also depends on what language is used (HTML will be lower, for instance).

MERN stack engineers will develop the applications using MongoDB, React, Node and Express JS.

Full Stack engineers can handle both front and back-end.

MongoDB is a NoSQL distributed database, used for projects that need to start immediately.

How Yahoo started: 25 years back, Yahoo started out as a website called "Jerry and David's guide to the World Wide Web", which was a directory of some of their favourite sites organized by topic. YahooChat was famous for messaging, along with chatrooms. Google prepared algorithms and ranking analysis, SEO, and users started using Google instead. Now, AI tells you things without you having to search for them and even suggests websites, which has led to people using it even over Google.

GEO - Generative Engine Optimisation

Software Crisis (1960s-1970s) - Software failures happened easily at this time, with just a single missing hyphen costing businesses thousands. Increasing complexity of computer programs, with the inability of existing methods and tools to keep pace. Projects were often over budget, late, unreliable, and failed to meet requirements.

TESTING

Testing levels:

- **Unit testing** done by developers to ensure logic works as per requirements. Separate, small modules.
- **Integration testing** for several units that are integrated and combined together.
Top-down approach (main program developed first, followed by sub-programs that branch off of the main program. requirements should be clear for this. dummy modules that have temporary values to be replaced later are called stubs),
bottom-up (sub-programs developed first, followed by the main program. used when requirements are not clear, so that sub-programs can all be integrated into the main program when things are clarified. can have clusters, which are groups of sub-programs that create a meaningful input only when executed together. instead of stubs, they're called drivers in bottom-up), **hybrid** (top-down and bottom-up combined, also known as sandwich integration. both stubs and drivers are used, but no clusters). **Big-bang testing** is for smaller applications where you wait until all units are complete and test everything at once.
- **System testing** for the entire software. High-level.
- **System-integration testing.** When shopping, you have the shopping site and the banking site, and the softwares have to be combined for the payment to happen. Physical connections too, like with printers or with HDMI cables or projecting.
- **User Acceptance Testing.** Customer does this.

Testing can be **manually** done by a human, or **automated** using tools such as Selenium or Playwright. Features that are stable and do not need further inputs to be tested will be automated and checked by tools, which will collect reports and pass them to the developers. Also, features that were released near launch, and features that need to be tested frequently and constantly. For newer features currently being developed, manual testing is definitely needed.

It can be further divided into functional and non-functional testing. **Functional** requirements require an input and give an output, focused on how a system is *doing* certain things. **Non-functional** requirements include things like how many simultaneous users a website can accommodate, and are centred around *performance and quality* of a software. UI/UX, cybersecurity, performance, etc. can all come under non-functional requirements.

Something can only be considered a bug if it goes against the requirements.

Verification (or quality control/static testing) is finding a blue pen (check document and see if something is being applied in the code, design, anywhere. Testing without executing).

Validation (or quality assurance/dynamic testing) is using the blue pen to make sure it's blue (in the end software, give inputs and see if it responds accordingly. Testing and getting an output).

CMM - Capability Maturity Model. Five levels exist. 1 lowest, 5 highest. For level 5, customer pays more because it's a high quality product or service.

ISO is proof that quality standards are being followed.

DEVELOPMENT MODELS

- **Waterfall Model:** Top to bottom, from level to level. You can't backtrack. Good for when there's no room for change, requirements should be crystal clear. Any changes will be costly and time-consuming, causing severe delays. Preferably for small applications.
- **V-Model:** Parallel testing for every phase. V stands for verification and validation. Look at types of testing above.
- **Iterative Model:** Dividing the software into chunks and taking care of each chunk before moving on.
- **Evolutionary Prototype Model:** Prototype created to see if things work and the customer is satisfied before actually spending money on the real version. Good for when the customer isn't sure exactly what they need and so precise requirements cannot be collected, or if requirements are evolving. After the requirements are clear, you can move from this to waterfall. Prototype is created to save time and is not necessarily made in the required language, the actual product is then recreated with the actual requirements.
- **Spiral Model:** Used when there are lots of risks. Divided into 4 sections that are repeated again with each iteration. Per module. First, you need to find multiple solutions to a single objective. See with the team which is the best to go with and what should be avoided. Identify risks for each solution and then develop. Review once it is ready, and plan for the next phase.
- **Agile Model:** Currently widely used. Iterable, easy to add changes due to flexibility, prioritizes communications. Deliverables are given for every phase. Scrum, Kanban, etc. For **scrum**, we have sprints that have a specific duration for something to be developed and deployed. Brief daily stand-ups (5-15 mins) are done to ensure no one stays blocked or idle and all team members can keep working. Blocks don't get resolved in the daily stand-up, someone volunteers to help after or the person gets referred to someone else. Scrum is mostly used for development projects. **Kanban** is used for support teams, when they get a ticket they develop and complete it immediately. Some companies use **hybrid scrumban** to handle development and ticketing together.

JIRA

- Epic (Purple): Big details of a project. Multiple user stories and tasks. Separate user stories for registration, log in, etc. A project can have multiple epics.
- Story (Green): Coding-related items. Multiple sub-tasks, which can be broken down according to developers. Connected to a parent epic to reduce searching.
- Task (Blue): Anything related to non-coding, such as documentation, cloud, reviewing, performance.
- Bug (Red): Self-explanatory, errors and bugs that need to be fixed.

AI runs on LLMs (Large Language Model).

TESTING (CONTINUATION)

Testing can be divided into:

- **White Box Testing:** Also called Glass Box Testing (GBT) or Transparent Box Testing (TBT). Done by an experienced developer, reviews the code. Checks whether unwanted variables are being declared and if memory is being wasted.
- **Black Box Testing:** Also called Closed Box Testing (CBT) or Opaque Box Testing (OBT). The code itself does not matter, nor is it known for the tester, but you know what output should be given.
- **Grey Box Testing:** Specific parts can be White Box Tested and other parts can be Black Box Tested to minimize the costs that come with WBT. Experienced developers only look at what is necessary for the specific project.

Black Box Testing Techniques:

- **Boundary Value Analysis (BVA):** Test upper limit and upper limit + 1, then lower limit and lower limit - 1. LL-1 and UL+1 should give errors.
- **Equivalence Partition:** Pick anything below the range, anything within the range, and anything above the range. Used when there is a set of classes. Example 1: Student grades. 0-50 should be a fail. Anything above 50 is assigned a grade such as A, B, C, etc. Example 2: Age condition. When booking tickets, people below 6 will have free tickets, people below 18 will have discounts, etc.
- **Error guessing:** Essentially just making educated guesses based on experiences. You know where there could be errors just by being familiar with someone else's coding.

Testing within the requirements is called **positive testing**. Testing outside of the requirements is **negative testing**.

Re-testing is testing a single part again after a bug has been fixed to ensure that the code works.

Regression testing checks whether there are new bugs due to an existing bug fix.

Requirement Traceability Matrix (RTM) shows corresponding requirements for a scenario and test case. When one test case fails, you can trace whatever is connected.

Writing test cases:

- Everyone used Excel in the past, now people use Zephyr, etc.
- There is a table with all requirements, and each requirement should have a name. Have unique scenario numbers, and scenario name and description. [Examples](#) spreadsheet with test cases, scenarios, and data.
- Scenarios are high level.

- Exact steps should be written for a test case.
- Expected and actual results should be the same for the status to be a pass. If they differ, status will be 'fail'.

Project manager breaks down the software document into different parts for specific team members to share the necessary requirements, so that members don't have to scroll through a 200+ page document needlessly.

FRS - Functional Requirement Specification.

SOFTWARE TESTING LIFE CYCLE

1. **Requirement Analysis:** Ensure requirements are known in a document that testers are going to use.
2. **Test Planning:** Planning is necessary to not waste time on things that do not need to be tested. Decide if you need manual or automated testing.
3. **Test Design:** Creating test scenarios with test cases and expected results.
4. **Environment Setup:** QA Environment is set up to be able to execute testing.
5. **Test Execution:** Test cases are executed, defects are logged and reported, and retesting is performed to confirm fixes.
6. **Test Closure:** Document is written that test cases have been passed, what bugs have been found and fixed, depending on high-medium-low priority.

PERFORMANCE TESTING

There exist performance testing tools known as LoadRunner, Blazemeter, Neoload, etc. that enable a website to load test with many users. Same tool can be used for load and stress testing, and you just need to find the breakage point where it stops working properly by gradually increasing.

Several types of performance testing:

- **Load testing:** Testing how quickly the site should load with a given amount of users. If a client specifies the software/website being able to load in 2 seconds with 5,000 people active, this is the requirement. This is testing for *expected* loads when the project goes live.
- **Stress testing:** Testing how much it can take before slowing down severely or crashing, and where the breaking point is. Knowing whether it's necessary to upgrade server size or optimize code. This is testing for *unexpected, heavy* loads when the project goes live.

Security testing: Ensuring that there is no unauthorized access to confidential or sensitive data. Some banks don't allow access from different countries because of this.

Bug Bounty: Some companies offer compensation from 500-1,000,000 dollars for any bugs that are found in their software.

Summary of sites we learned how to use:

- GitHub (how to create a repository and upload an html file for a website)
- Jira (how to create an epic and stories)
- Cowboy Prompt alongside Claude AI or ChatGPT for smarter prompts.